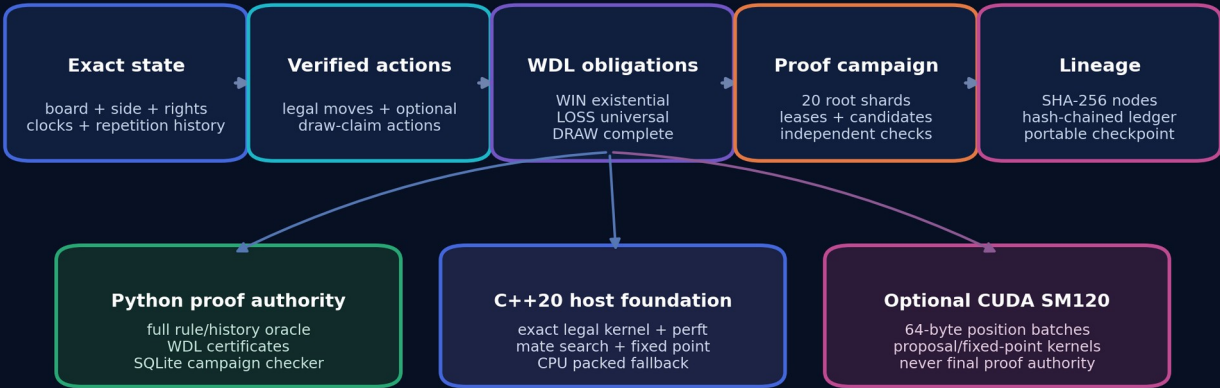


UGTS CHESS GAME-THEORETIC SOLVER 2.0

Classical proof campaign + exact WDL certificates + RTX 5070 Ti Laptop execution profile



Captured root status: UNKNOWN - 20 exact root obligations; 0 independently verified child WDL values

The GPU accelerates proposals. The checker decides what is true.

Release field	Value
Canonical identity	ugts.application.chess-proof@2.0.0
Release	UGTS Chess Game-Theoretic Solver 2.0
Target	NVIDIA GeForce RTX 5070 Ti Laptop GPU, nominal 12 GB GDDR7
Captured root value	UNKNOWN
Document date	29 August 2026
Status	Executable host-validated foundation; CUDA laptop build and long campaign delegated to Codex/device

EVIDENCE BOUNDARY

This release attempts a proof-oriented solution of classical chess. It does not claim that the standard initial position is weakly, strongly or ultra-weakly solved. GPU kernels propose work; independently checked certificates decide truth.

Executive Summary. Release Decision

What has been built, what has been proved, and what remains open.

FORMAL DECISION - GO

Promote Chess 2.0 as an upgraded foundational proof campaign and RTX 5070 Ti Laptop handoff. Freeze the exact rule/history identity, represent claimable draws as actions, issue content-addressed WDL obligations, and keep the classical initial position UNKNOWN until quantified proof obligations close.

Verified release result

Area	Captured result
Initial position	20 exact root obligations; 0 independently verified child values; root UNKNOWN
Python	92 tests passed; 20 exact perfct checks
Native C++20	3 CTest gates passed; depth-5 initial perfct 4,865,609 nodes
Packed protocol	4 positions; 97 moves; 0 differential mismatches
Certificates	Mate certificate verified; exact bounded WIN fixture; initial-depth-2 fixture safely UNKNOWN
Tablebases	Complete KQK max DTM 20 plies; KRK max DTM 32 plies
Distribution	Wheel 332,019 bytes; isolated clean install passed
CUDA laptop	Source and SM120 preset supplied; no physical laptop benchmark claimed in this environment

What the deliverable is for

- Codex can compile the optional CUDA adapters, compare them against the exact Python and C++ host oracles, and record device evidence without weakening proof semantics.
- The twenty first moves can be leased as independent shards, with candidate WDL results kept non-authoritative until a separate checker record is attached.
- The package supplies a reproducible starting point for out-of-core proof DAGs, larger exact endgame partitions, dead-position certificates and distributed replay.

Reading Map. Contents and Claim Ladder

The report moves from proof meaning to exact state, certificates, campaign orchestration, RTX execution and promotion gates.

Sections	Purpose
1-3	Meaning, source basis and component identity
4-7	FIDE action profile, UGTS authority and exact history state
8-12	WDL calculus, certificates, root obligations and SQLite campaign
13-15	Bounded exact results, retained mate proof and exact KQK/KRK tables
16-20	C++20 core, packed protocol, RTX profile, VRAM plan and CUDA boundary
21-24	Codex workflow, global proof architecture, validation and performance evidence
25-28	Roadmap, mechanism catalog, package map, final definition and references

What Does 'Solve Classical Chess' Mean?

The package keeps rule correctness, solved positions and a solved initial game visibly separate.



Weak, strong and ultra-weak solution terminology describes proof scope; none is claimed here.

Figure 1. Rules solved, positions solved and the initial game solved are separate claims. Chess 2.0 establishes the first and selected instances of the second; the third remains UNKNOWN.

READING RULE

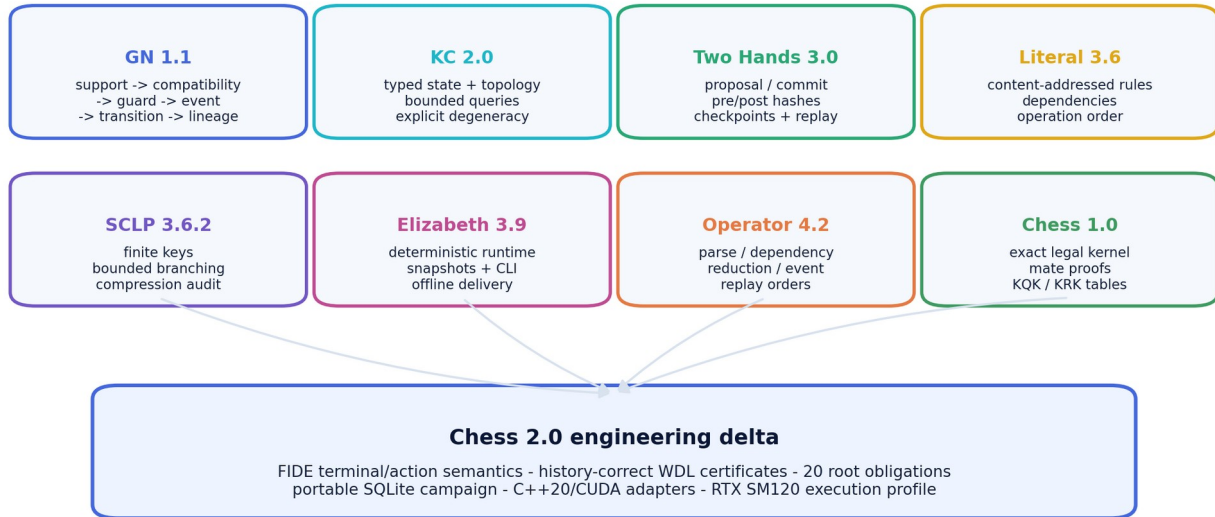
An exact result is promoted only when its serialized rule profile, state identity, action coverage, certificate graph and independent verification status support it. Search depth, evaluation score and GPU completion do not substitute for those obligations.

1. Source Basis and Component Identity

UGTS supplies authority discipline; chess rules, WDL logic and GPU engineering are explicit application additions.

Source Grounding and Engineering Delta

UGTS supplies the authority discipline. Chess rules, WDL logic and CUDA work are application engineering.



SARA 3.6.3 was reviewed but its wallet-specific mechanisms were excluded from the chess model.

Figure 2. The new chess layer retains the recurring support -> compatibility -> guard -> verified event -> transition -> lineage architecture.

1.1 Immediate source use

Source	Use in Chess 2.0
UGTS-GN 1.1	Authority chain, packed precision and hardware evidence boundaries
UGTS-KC 2.0	Typed state, topology, bounded queries and explicit degeneracy
Two Hands 3.0	Proposal/commit, pre/post hashes, checkpoints and replay
Literal 3.6 / SCLP 3.6.2	Content addresses, dependency order, finite keys and bounded branching
Elizabeth 3.9	Deterministic game world, snapshots, CLI and offline delivery
Go 4.2 / Operator 4.2	Rules-position-game claim ladder; exact certificates; distinct evaluation, event and replay orders
Chess 1.0	Exact legal kernel, mate proofs, search boundary and complete KQK/KRK tables

Grounding: The source PDFs do not claim a classical chess solution or RTX implementation. The chess/WDL/campaign/CUDA layers are visibly engineering-derived. SARA 3.6.3 was reviewed but its wallet-specific mechanisms were excluded.

2. What a Game-Theoretic Solution Requires

A finite game can be defined exactly while its initial value remains computationally unresolved.

2.1 Three independent claims

Claim	Required evidence	Chess 2.0 status
Rules solved	Every legal action, terminal and transition is deterministic and testable.	Implemented
Position solved	A fully serialized finite state has a complete proof certificate or exact table entry.	Implemented for bounded fixtures, mate certificates, KQK and KRK
Game solved	The orthodox initial position is certified WIN, DRAW or LOSS under one complete rule profile.	Not established; UNKNOWN

2.2 Weak, strong and ultra-weak terminology

- An ultra-weak solution determines the game-theoretic value of the initial position.
- A weak solution also gives a strategy from the initial position that achieves that value against every reply.
- A strong solution gives the game-theoretic value and optimal action for every legal position in the selected state space.
- Chess 2.0 claims none of these for unrestricted classical chess. Its root node is deliberately serialized as UNKNOWN.

WHY UNKNOWN IS A RESULT

UNKNOWN means one or more quantified obligations remain open. It is not a heuristic draw, an engine disagreement, a timeout disguised as a result, or evidence that no solution exists.

3. Classical FIDE Rule and Action Profile

Moves, claimable draws and automatic terminals are distinct action classes.

Action/terminal	Class	Proof treatment
Ordinary move	Legal board transition	Must pass occupancy, motion, special-right and king-safety checks
Current draw claim	Optional player action	Threefold repetition or 50-move condition already present
Claim by intended move	Optional player action	Declared legal move would create third repetition or complete 50 moves
Checkmate	Automatic terminal	Side to move has no legal move and is in check; winner is the opponent
Stalemate	Automatic terminal	No legal move and not in check
Fivefold repetition	Automatic terminal	Exact repetition identity occurs at least five times
75-move rule	Automatic terminal	150 halfmoves without pawn move or capture; checkmate takes precedence
Dead position	Automatic terminal	Requires an exact certificate that no mating sequence is possible

3.1 Declared profile

```
rules_profile = fide-classical-2023-claims-as-actions-v2
move claims are optional actions
automatic draws are terminals
checkmate is evaluated before the automatic 75-move terminal
```

DEAD-POSITION BOUNDARY

The bundled recognizer certifies a conservative exact subset. A global classical proof must add a complete dead-position oracle or attach independently checkable dead-position certificates. Uncovered cases remain UNKNOWN.

Grounding: FIDE Laws of Chess, Articles 5.2, 9.2, 9.3 and 9.6. The official 2023 text states that checkmate takes precedence over the automatic 75-move draw.

4. One Action Through the UGTS Authority Chain

A move generator proposes; only the verifier and atomic transition alter authoritative state.

One Chess Action Through the Canonical UGTS Authority Chain



Figure 3. Chess mechanics mapped onto the canonical six-stage authority sequence.

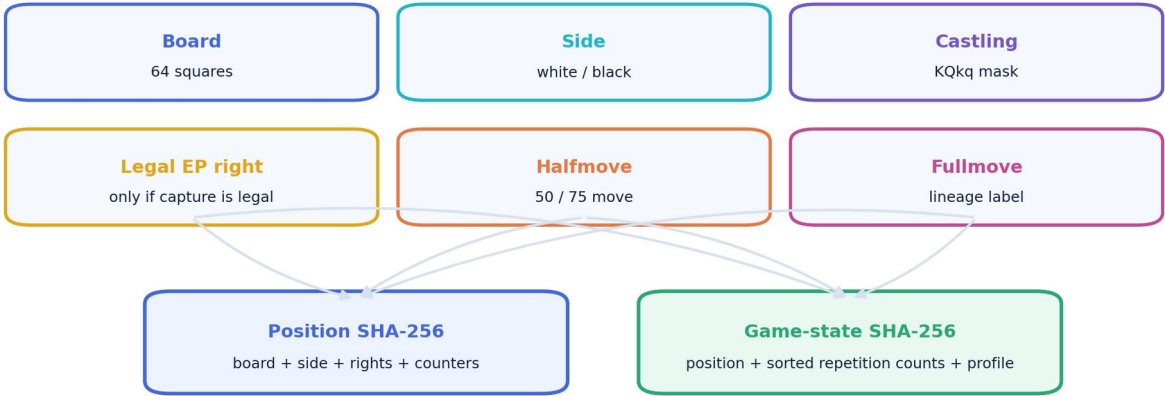
Stage	Chess interpretation
WHERE / support	Rays, jump targets, pawn attack/move supports, king neighborhood
MATCH / compatibility	Correct side, piece type, target occupancy and capture relationship
READY / special guards	Promotion choice, castling rights/path, legal en-passant right
CHECKED / verified event	Move leaves own king safe; terminal classification follows the new state
CHANGE / transition	One immutable patch updates board, side, rights, en-passant and clocks
REMEMBER / lineage	Exact repetition counts, state hashes, move event and replay order
NON-BYPASS RULE CPU search, CUDA kernels, a human interface, opening book, tablebase adapter or neural policy may order or propose actions. None may set an authoritative WDL value without the same legal and certificate checks.	

5. Exact Position and History Identity

A board diagram is not enough to decide repetition, move-count claims or proof replay.

Exact Identity: Position Is Not Complete Game State

Repetition, move-count claims and proof replay require explicit history context.



A 64-bit key is permitted as an index. It is never accepted as the complete proof identity.

Figure 4. The proof identity includes normalized rule state and exact repetition-count context.

5.1 Authoritative state

Q = (board, side, castling rights, legal en-passant right, halfmove clock, repetition-count context, rule profile, lineage)

Identity	Role
Position SHA-256	Exact FEN-level transport identity including fullmove metadata
Game-state SHA-256	Rule-semantic position plus capped halfmove clock, sorted exact repetition counts and profile ID
64-bit cache key	Permitted only as a fast index; full record must be compared before proof use
Lineage hash	Ordered event evidence; not interchangeable with current-state identity

Grounding: This preserves the recurrent UGTS distinction between coordinates, identity and lineage. SCLP finite keys are indexes, not complete worlds.

6. Repetition, Move Counts and Draw Strategy

A claim opportunity changes available actions but need not end a winning line.

6.1 Exact action semantics

Condition	Record code	Game-theoretic treatment
Threefold now	claim_threefold_current	Player may take DRAW or continue
50 moves now	claim_fifty_move_current	Player may take DRAW or continue
Third repetition by move	claim_threefold_by_move	Claim is attached to the declared legal move
50 moves by move	claim_fifty_move_by_move	Claim is attached to the declared legal move
Fivefold	fivefold_repetition	Forced DRAW terminal
75 moves	seventy_five_move	Forced DRAW terminal after checkmate test

6.2 Repetition identity

- Board occupancy, side to move and castling rights must match.
- An en-passant square contributes only when an en-passant capture is actually legal; a merely printed target is normalized away.
- Exact occurrence counts are stored as sorted SHA-256 identity/count pairs. A rolling checksum alone is insufficient.
- Fullmove number is replay metadata and does not change legal action or terminal rules; the semantic proof hash excludes it.

STRATEGIC CONSEQUENCE

At halfmove 100, a player may already claim a draw. If the same position also contains checkmate in one, WIN remains exact because the player can choose the winning move instead of the optional draw action.

7. Four-Valued WDL Proof Calculus

WIN, LOSS and DRAW are proof claims; UNKNOWN is the safe incomplete value.

Four-Valued WDL Proof Calculus

Values are always from the side-to-move perspective; cutoffs stay UNKNOWN.



Claimable threefold/50-move draws are optional actions. Checkmate outranks the automatic 75-move rule.

No-mate-within-N, an engine evaluation of 0.00, or a GPU fixed point on an incomplete graph is not a draw proof.

Figure 5. Values are from the side-to-move perspective. Child values are inverted when interpreted by the parent.

7.1 Quantified obligations

```
WIN(q) := exists legal action a with exact LOSS(child(q,a))
LOSS(q) := every legal action is covered and exact WIN(child(q,a))
DRAW(q) := no winning action exists, coverage is complete, and a draw terminal/action/closed-complement certificate exists
UNKNOWN := any required edge, history state, dead-position decision or checker obligation is open
```

CUTOFF LAW

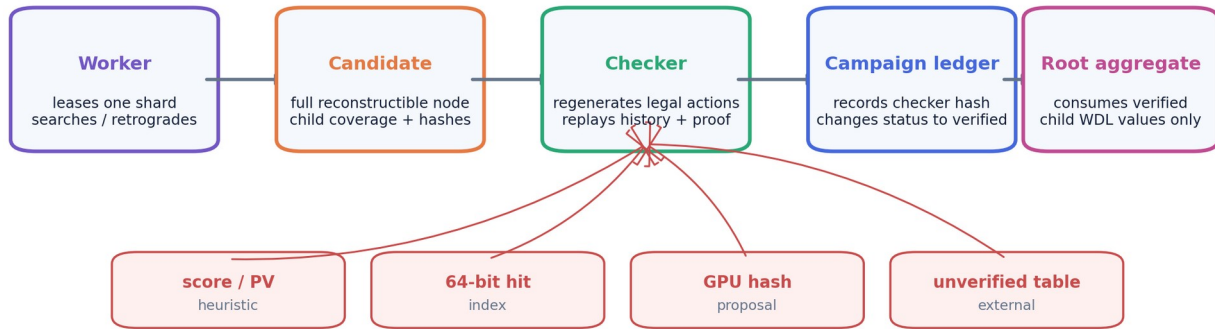
A horizon, node budget, time limit, missing child or heuristic score is UNKNOWN. The solver never promotes 'no win found yet' to DRAW.

Grounding: The fixed-point structure follows the bounded exact-solving precedent in UGTS-KC 4.2 Go and the order/certification discipline in the General Operator and Order Addendum.

8. Content-Addressed Certificates and Independent Promotion

A worker result is evidence only after a separate checker validates quantified coverage.

Candidate Certificate and Independent Promotion



Promotion is an event with evidence; a worker never writes authoritative WDL directly.

Figure 6. A certificate path and a checker record are both bound into campaign lineage.

Record	Required content
Node identity	Game-state SHA-256, FEN, history counts, remaining depth and rule profile
Child obligation	Action kind, UCI/SAN or claim code, child state hash, child value and child certificate hash
Coverage	terminal, witness, complete or cutoff
Certificate hash	SHA-256 over canonical node content excluding the hash field itself
Checker record	Obligation ID, claimed WDL, certificate SHA-256, child game-state SHA-256 and valid=true
Promotion	Campaign job changes to verified only after checker fields and hashes match

INDEPENDENCE BOUNDARY

The current reference checker runs in the same Python package but through a separate verification path and record. A production campaign should additionally diversify implementations, hosts and reviewers before treating a root certificate as durable evidence.

9. The Initial Position Becomes Twenty Root Obligations

The first exact decomposition is small, stable and independently leaseable.

The Classical Initial Position Becomes 20 Exact Root Obligations

Each child has a full FEN, position hash, game-state hash and repetition-count record.

01 a2a3 depth-4 paths: 8,457 WDL: UNKNOWN	02 a2a4 depth-4 paths: 9,329 WDL: UNKNOWN	03 b1a3 depth-4 paths: 8,885 WDL: UNKNOWN	04 b1c3 depth-4 paths: 9,755 WDL: UNKNOWN	05 b2b3 depth-4 paths: 9,345 WDL: UNKNOWN
06 b2b4 depth-4 paths: 9,332 WDL: UNKNOWN	07 c2c3 depth-4 paths: 9,272 WDL: UNKNOWN	08 c2c4 depth-4 paths: 9,744 WDL: UNKNOWN	09 d2d3 depth-4 paths: 11,959 WDL: UNKNOWN	10 d2d4 depth-4 paths: 12,435 WDL: UNKNOWN
11 e2e3 depth-4 paths: 13,134 WDL: UNKNOWN	12 e2e4 depth-4 paths: 13,160 WDL: UNKNOWN	13 f2f3 depth-4 paths: 8,457 WDL: UNKNOWN	14 f2f4 depth-4 paths: 8,929 WDL: UNKNOWN	15 g1f3 depth-4 paths: 9,748 WDL: UNKNOWN
16 g1h3 depth-4 paths: 8,881 WDL: UNKNOWN	17 g2g3 depth-4 paths: 9,345 WDL: UNKNOWN	18 g2g4 depth-4 paths: 9,328 WDL: UNKNOWN	19 h2h3 depth-4 paths: 8,457 WDL: UNKNOWN	20 h2h4 depth-4 paths: 9,329 WDL: UNKNOWN

Exact depth-4 legal workload: 197,281 leaf paths | verified child values: 0 | root: UNKNOWN

Figure 7. Every legal first move has a stable obligation ID, child state hashes and an exact depth-4 workload count.

Largest shard	Move	Depth-4 leaf paths	Captured WDL
root-12-e2e4	e2e4	13,160	UNKNOWN
root-11-e2e3	e2e3	13,134	UNKNOWN
root-10-d2d4	d2d4	12,435	UNKNOWN
root-09-d2d3	d2d3	11,959	UNKNOWN
root-04-b1c3	b1c3	9,755	UNKNOWN
root-15-g1f3	g1f3	9,748	UNKNOWN
root-08-c2c4	c2c4	9,744	UNKNOWN
root-05-b2b3	b2b3	9,345	UNKNOWN

EXACT FIRST CHECKPOINT

The twenty shard workloads sum to 197,281, exactly matching standard initial-position perft depth 4. This checks the decomposition; it does not determine any child's full-game value.

10. Root Aggregation and Current Campaign Status

Only independently verified child values participate in the initial-position result.

Root value	Complete quantified rule
White root WIN	At least one verified child is LOSS for Black to move
White root LOSS	All twenty verified children are WIN for Black to move
White root DRAW	All twenty are resolved, none is LOSS for Black, and at least one is DRAW
White root UNKNOWN	Any required child is unverified or UNKNOWN, unless a verified LOSS child already proves WIN

10.1 Captured ledger state

Metric	Captured value
Root obligations	20
Verified child values	0
Root value	UNKNOWN
Event hash chain	valid
Depth-4 leaf-path checkpoint	197,281
child values are from the child side-to-move perspective one child LOSS -> root WIN all children WIN -> root LOSS resolved mix containing DRAW and no LOSS -> root DRAW otherwise -> root UNKNOWN	

CURRENT FORMAL RESULT

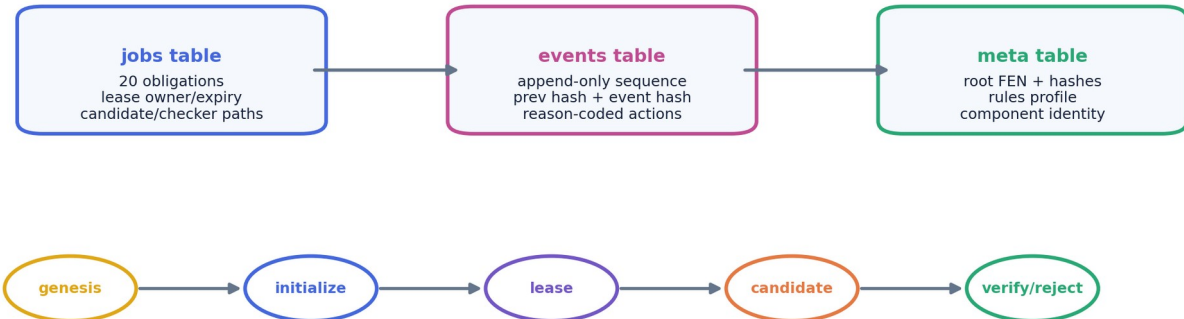
The standard initial position is UNKNOWN in this package. No principal variation, engine score or opening preference is presented as a substitute.

11. Portable SQLite Proof Campaign

Coordination state is mutable; proof records and the event journal are content-addressed and auditable.

Portable SQLite Campaign Ledger

Coordination state is mutable; proof records and event hashes make changes auditable.



Campaign files use paths relative to the database directory, so the checkpoint verifies after extraction on another machine.

Captured ledger: 20 unresolved jobs, 1 initialization event, valid hash chain, root UNKNOWN.

Figure 8. Jobs, events and metadata are separated; path references are package-relative so the campaign remains portable.

Element	Contract
meta	Root FEN/hashes, rule profile, schema and initialization data
jobs	20 obligations, lease owner/expiry, attempts, candidate WDL, certificate and checker paths/hashes
events	Append-only sequence, canonical payload, previous hash and event hash
Lease	BEGIN IMMEDIATE transaction selects one unresolved or expired job
Candidate	WIN/DRAW/LOSS plus certificate file hash; UNKNOWN cannot be submitted as a candidate
Verification	Separate JSON checker record must match obligation, value, certificate and child state

RELOCATION TEST

The database and its relative shard/certificate/checker paths were copied to another temporary location and reverified. This guards against absolute-path evidence that works only on the packaging machine.

12. Bounded Exact Results and Safe Incompleteness

Two fixtures demonstrate existential closure and honest UNKNOWN propagation.

Two Bounded Results, Two Different Proof Statuses

Mate Overrides Optional Claim

Position KQK, halfmove clock 100

Available action claim 50-move draw

Winning witness Qa4#

Result **WIN**

nodes: 2 exact: true

Initial Position, Depth 2

Root legal moves 20

Searched nodes 421

Cutoffs 400

Result **UNKNOWN**

No cutoff is relabeled as DRAW.

The first certificate closes an existential proof. The second records useful work without overstating what it proves.

Figure 9. The first fixture is exactly WIN despite an optional draw claim; the second remains UNKNOWN at the ply horizon.

Fixture	Root	Plies	Nodes	Value	Reason
Mate over claim	8/8/8/1Q6/8/8/k7/2K5 w -- 100 2	1	2	WIN	Qa4# is an exact witness; draw claim is optional
Initial position	standard initial FEN	2	421	UNKNOWN	400 depth-2 leaf cutoffs remain open

12.1 Certificate hashes

mate root certificate: 87d8a14d9b5af5cbfbccb9522f9c6ae372b66f68992c8749667a80dd1962dfb7

mate child certificate: 891172e29d5aca1a49c857b4c2080284a98b717507e394061efd1f1f3ecb266f

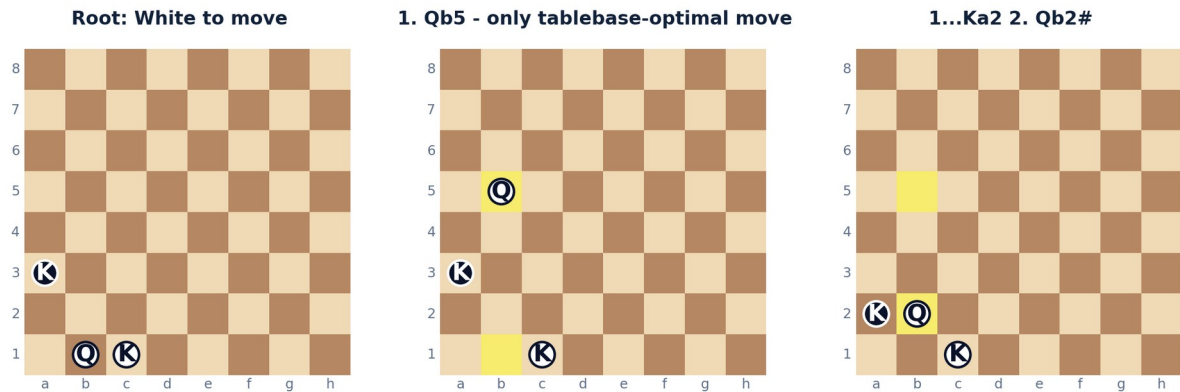
PROOF LESSON

An existential WIN can close early when one exact losing child is found. Universal LOSS and complete DRAW cannot close until all relevant actions and history states are covered.

13. Retained Forced-Mate Certificate

Chess 1.0's OR/AND mate proof remains a compact independently verifiable exact layer.

Worked Exact Result: Mate in Two



FEN root: 8/8/8/8/k7/8/1QK5 w - - 0 1 | KQK tablebase DTM = 3 plies

Figure 10. Worked certificate: 1. Qb5 Ka2 2. Qb2# from the declared KQK fixture.

Proof Certificate Semantics

OR nodes choose one proved attacker move; AND nodes enumerate every legal defender reply; leaves must verify as checkmate.

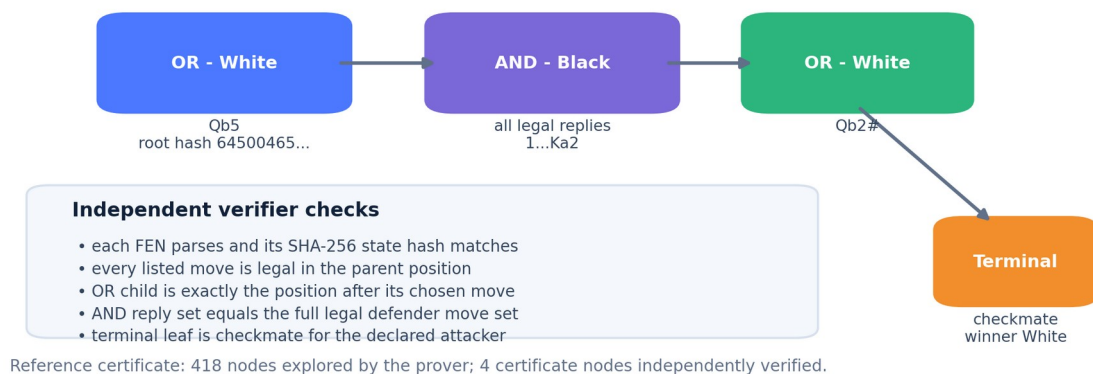


Figure 11. The attacker's OR node supplies one winning move; the defender's AND node enumerates every legal reply.

EXACT SCOPE

This proves a forced mate from one serialized position within the declared horizon. It is not evidence about the game-theoretic value of the 32-piece initial position.

14. Complete KQK and KRK WDL/DTM Tables

Two finite three-piece state spaces are bundled as exact, reconstructible proof partitions.

Bundled Exact Three-Piece WDL/DTM Tables

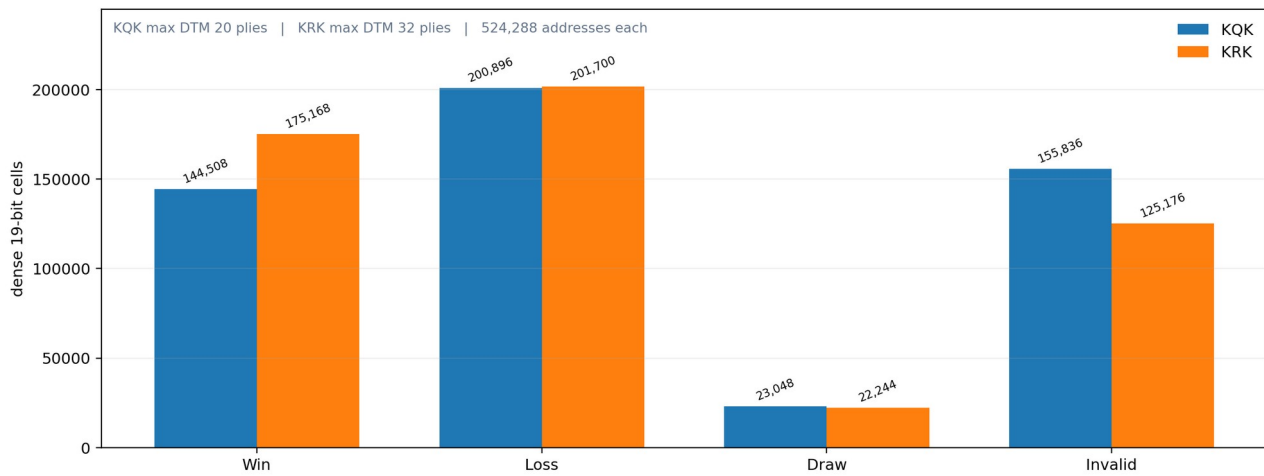


Figure 12. All 2^{19} packed addresses are classified as valid WIN/LOSS/DRAW or invalid placement.

Table	Valid	WIN	LOSS	DRAW	Max DTM	gzip bytes
KQK	368,452	144,508	200,896	23,048	20	144,368
KRK	399,112	175,168	201,700	22,244	32	152,306

TABLEBASE BOUNDARY

Outcome is from the side-to-move perspective. DTM is plies to checkmate under optimal play. These two tables are complete for their declared material signatures, not a substitute for all 3- through 7-piece endgames.

15. Finite Addressing, Generation and External Adapters

Exact tables are proof partitions only when key, decode, legality and rule-profile contracts remain explicit.

Contract	Exact requirement
Address width	19 bits; 524,288 addresses for king, king, major piece and side-to-move fields
Validity	Overlapping pieces, adjacent kings and illegal check configurations are marked invalid
Generation	Retrograde fixed point seeds checkmates/stalemates and propagates side-to-move WDL/DTM
Transport	gzip payload plus JSON metadata, file length and SHA-256
Probe	Decode address, revalidate material/profile and return WDL/DTM
External adapter	May query larger Syzygy partitions, but must label rule profile and independently recheck transitions

15.1 Why these tables matter to a global attempt

- Solved material partitions become terminal proof nodes for larger predecessor graphs.
- Distance metadata supports deterministic witness extraction and cycle avoidance.
- Hashes permit distributed workers to refer to one exact table version rather than an ambiguous service response.
- A full campaign still needs promotion/capture transitions, 50/75-move policy compatibility and history-aware boundaries.

COMPRESSION RULE

Smaller storage is useful only with a total decoder and exact outcome equivalence. The tablebase key is a finite address, not a claim that chess state is one bit or that all history has disappeared.

16. Native C++20 Foundation

A second implementation provides fast exact checks and a clean CUDA integration boundary.

Native C++20 Foundation and Its Boundary

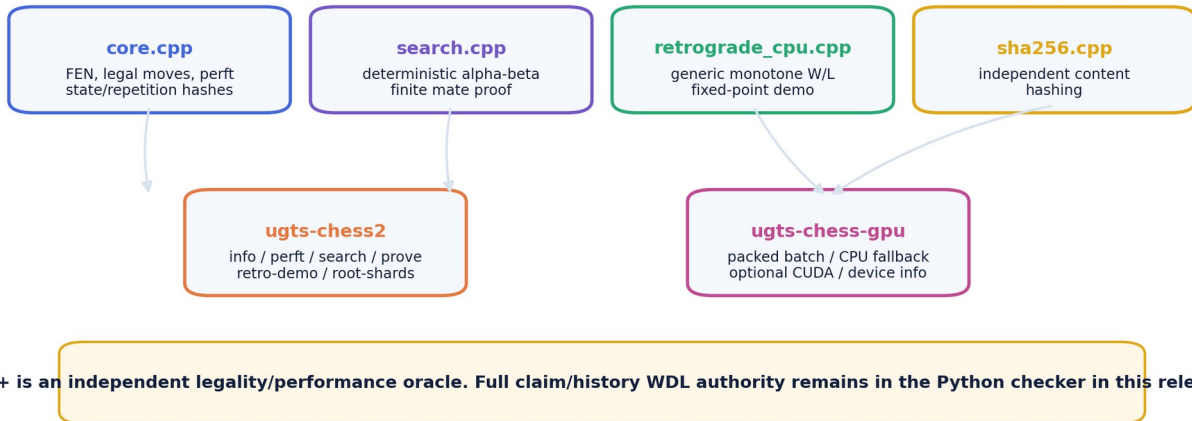


Figure 13. Host legality and performance oracles are separate from the Python history/WDL authority.

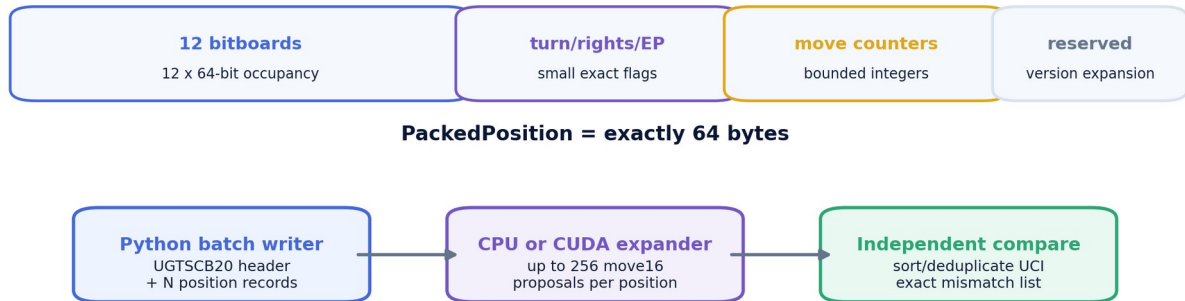
Module	Role
core.cpp / fen.cpp	Board representation, attacks, legal transitions, perft and notation transport
search.cpp	Deterministic alpha-beta and bounded mate search; scores are not global proof
retrograde_cpu.cpp	Generic monotone WIN/LOSS fixed-point demonstration
sha256.cpp	Dependency-free content hashing for native records
ugts-chess2	info, selftest, perft, search, mate, root-shards and retro-demo
ugts-chess-gpu	device-info, packed batch generation and CPU/CUDA self-test
CROSS-IMPLEMENTATION PURPOSE The C++ layer is not merely a speed port. It provides a second executable implementation for perft, move proposals and fixed-point behavior, enabling differential checks before CUDA results are admitted.	

17. Packed Position and Move Proposal Protocol

The wire format is simple enough to inspect, batch and compare across CPU and CUDA backends.

Packed Proposal Protocol

The same binary batch is executed by CPU fallback or CUDA, then compared with the Python oracle.



Captured host fallback: 4 positions, 97 proposed moves, 97 verified moves, 0 mismatches.

Figure 14. The simple kernel emits pseudo/legal candidates into a versioned binary batch; the Python oracle compares exact legal outputs.

Item	Size/class	Contract
PackedPosition	64 bytes	12 bitboards plus turn/rights/en-passant/clock/reserved fields
Packed move	16 bits	6 source bits, 6 target bits, 3 promotion bits, spare version space
Input file	UGCB	Header, record size/count and contiguous position records
Output file	UGMV	Per-position counts plus up to 256 encoded moves each
Ordering	Deterministic	Sorted/normalized before differential comparison
Authority	Proposal only	Python exact legal moves remain the captured comparison oracle

CAPTURED FALLBACK

The host CPU backend processed 4 positions and 97 moves with zero mismatches. CUDA compilation and physical-device comparison remain pending.

18. RTX 5070 Ti Laptop Execution Profile

Official product identity is a starting constraint; runtime inspection is the authoritative device record.

Profile field	Value
Product	GeForce RTX 5070 Ti Laptop GPU
Architecture family	NVIDIA Blackwell GeForce laptop
CUDA cores	5,888
AI TOPS	992
Standard memory	12 GB GDDR7
CUDA compute capability	12.0
Native target	sm_120 / compute_120
Minimum package preset	CUDA Toolkit 12.8 or newer; CUDA 12.8 added compiler support for SM120

18.1 Runtime gate

```
nvidia-smi --query-gpu=name,memory.total,memory.free,compute_cap --format=csv
ugts-chess-gpu device-info
nvcc --version
```

- Laptop power limits, clocks, free VRAM, cooling and driver/toolkit combinations vary by model and power mode.
- The build preset targets SM120. Datacenter Blackwell SM100a architecture-conditional kernels are not assumed compatible with GeForce RTX 50-series SM120.
- The package records the actual device name, compute capability, memory, SM count, warp size and maximum threads before a benchmark is promoted.

Grounding: Official NVIDIA GeForce RTX 50 Series laptop specifications, CUDA GPU Compute Capability mapping, CUDA 12.8 release notes, Blackwell compatibility guide and CUTLASS architecture note.

19. VRAM Budget and Out-of-Core Design

The nominal 12 GB frame buffer is a bounded cache, not a complete home for the chess proof graph.

RTX 5070 Ti Laptop 12 GB Starting Memory Plan

Runtime free-memory inspection overrides nominal capacity. Allocation failure reduces the batch.

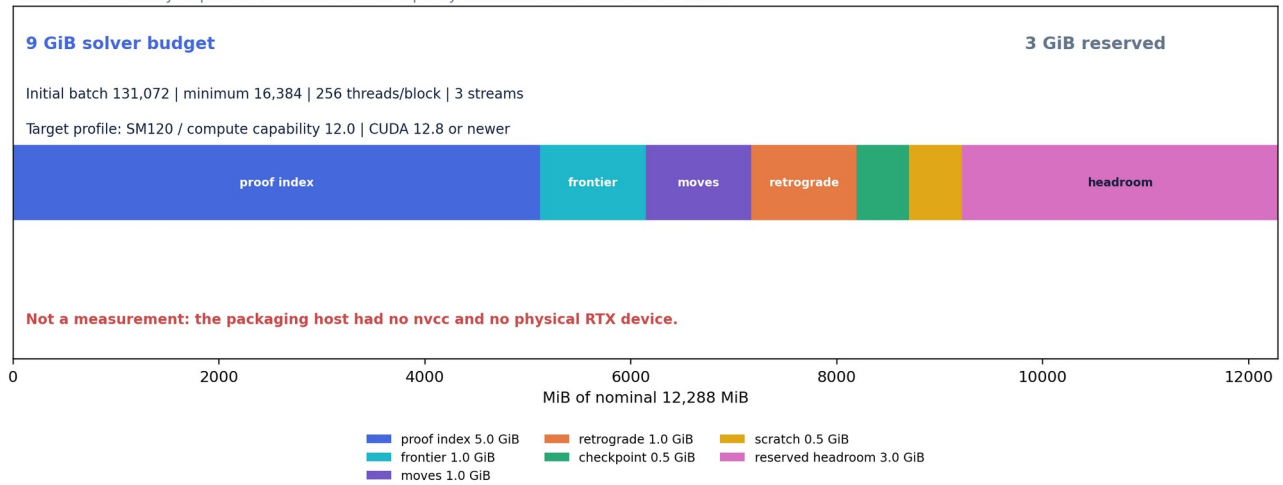


Figure 15. The checked-in starting profile reserves 3 GiB and assigns 9 GiB to solver structures.

Allocation	Starting size	Purpose
Proof/transposition index	5,120 MiB	Content-addressed lookup and verified-state cache
Frontier	1,024 MiB	Current batch of unresolved positions
Move buffers	1,024 MiB	Counts and 16-bit proposal arrays
Retrograde	1,024 MiB	Outcome vectors and graph work arrays
Checkpoint	512 MiB	Device-to-host export and integrity staging
Scratch	512 MiB	Temporary scans, prefix sums and error safety
Reserved headroom	3,072 MiB	Driver/display/fragmentation/thermal fallback margin

CAPACITY REALITY

Even 9 GiB divided by the 64-byte exchange record holds at most about 151 million records before indexes, move arrays, histories and certificates. A proof-scale campaign must stream, partition, checkpoint and persist to disk.

20. CUDA Kernels and Correctness Boundary

The first GPU code is deliberately simple so that equal-output validation comes before optimization.

Component	Current implementation
Scalar batch expander	One packed position per thread; fixed 256-move slot; output compared with host oracle
Generic retrograde step	One graph node per thread; monotone UNKNOWN -> WIN/LOSS propagation
Device inspector	Queries name, compute capability, total/free memory, SMs, warp and block limits
CPU fallback	Runs the identical batch protocol when CUDA is absent or rejected
CMake profile	UGTS_ENABLE_CUDA=ON; CMAKE_CUDA_ARCHITECTURES=120
Promotion gate	No mismatches, deterministic record order, full reconstructibility and independent proof checker

20.1 Optimization sequence after correctness

1. Measure scalar kernel occupancy, transfer overhead and batch-size behavior on the actual laptop.
2. Add warp-cooperative attack generation and prefix-sum compaction only behind the same binary protocol.
3. Separate hot structure-of-arrays data from the 64-byte exchange record while retaining a total versioned decoder.
4. Add asynchronous streams and double buffering; preserve deterministic per-position move ordering at the comparison boundary.
5. Keep WDL certificate checking on an independent host path until a separately verified GPU checker exists.

PACKAGING EVIDENCE

The delivery host had no nvcc and no physical RTX 5070 Ti. CUDA throughput, VRAM, power, temperature, clocks and long-run stability are intentionally unclaimed.

21. Codex Build, Measure and Promotion Workflow

The supplied scripts preserve device evidence under validation/device rather than overwriting host evidence.

Codex Promotion Workflow on the Laptop

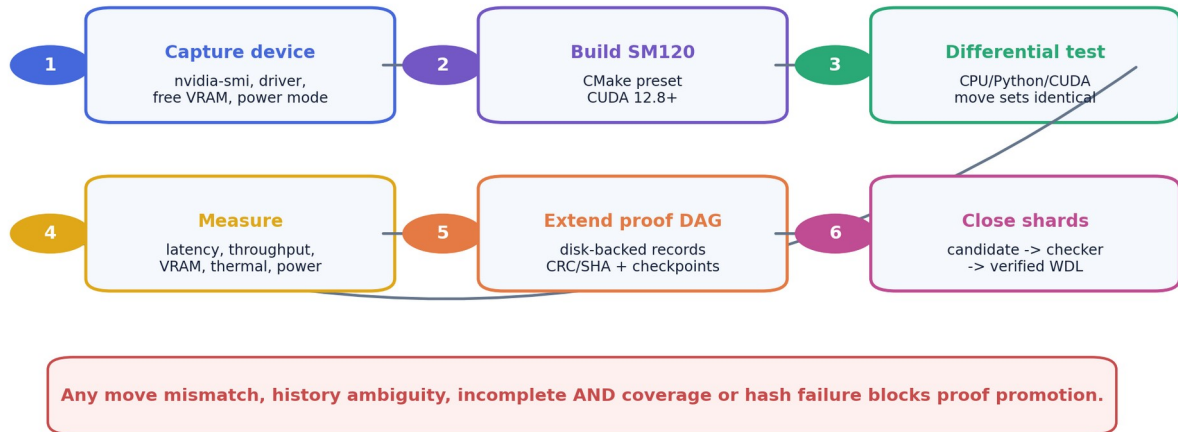


Figure 16. Device capture, SM120 build, differential testing, measurement, proof-DAG extension and shard closure are separate gates.

21.1 First commands on Windows

```
powershell -ExecutionPolicy Bypass -File scripts/build_rtx5070ti.ps1
powershell -ExecutionPolicy Bypass -File scripts/run_codex_campaign.ps1
```

21.2 Required measurements

- p50/p95/p99 batch latency, positions/s, moves/s and transfer/compute breakdown
- peak VRAM, host RAM and storage bytes per verified node
- 5-, 15- and 30-minute clocks, temperature, power, fallback and throttling behavior
- differential mismatch count across packaged fixtures and a seeded random legal corpus
- checkpoint/restart reproducibility and event-chain validation after process interruption

RELEASE BLOCKER

Any extra move, missing move, history ambiguity, incomplete AND-node reply set, corrupt checkpoint, hash mismatch or device-dependent result blocks proof promotion.

22. Path from Laptop Experiment to Global Proof Campaign

Progress is measured by independently verifiable obligation closure, not by engine rating or searched nodes alone.

Path Toward a Game-Theoretic Result

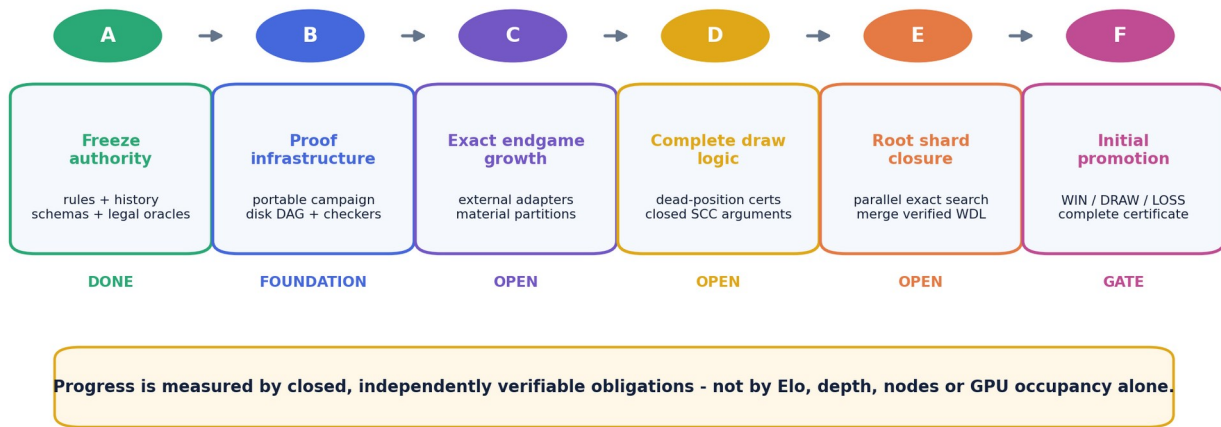


Figure 17. The initial node is promoted only after proof infrastructure, exact coverage and checker diversity are complete.

22.1 Disk-backed proof DAG

- Partition by material signature, rights/history class and canonical hash prefix; store full reconstructible state beside any short index.
- Use append-only frontier and certificate objects with schema IDs, CRC/SHA checks and content-addressed child references.
- Treat opening books, engines and neural policies as prioritizers. They cannot close a proof node.
- Integrate exact tablebase partitions through profile-labeled adapters and boundary transition checks.
- Merge root shards only after independent checker records agree; retain conflicts and rejected candidates in lineage.

SCALE BOUNDARY

A single 12 GB laptop can validate kernels, close small partitions and contribute shards. It is not represented as sufficient by itself to exhaust the full classical game graph.

23. Captured Host Validation

The supplied evidence establishes exact host behavior and packaging integrity, not physical-GPU performance.

Captured Host Validation Dashboard



Schemas: 26 records across 5 schema files | clean-install wheel: 332,019 bytes | mate certificate: 4 nodes verified

Evidence boundary: host timing is environment-specific; CUDA compile and physical-laptop measurements are pending.

Figure 18. All captured host gates passed; the CUDA device lane remains explicitly pending.

Gate	Count/artifact	Status
Python tests	92	PASS
Python perft checks	20	PASS
C++ CTest	3	PASS
Native packed self-tests	9	PASS
Schema documents	26	PASS
Mechanism records	104	PASS
Wheel clean install	ugts_kc_chess-2.0.0-py3-none-any.whl	PASS
Physical RTX run	0	PENDING

VALIDATION MEANING

This validates the supplied attempt and proof machinery; it does not solve the 32-piece initial position.

24. Performance Evidence and Non-Claims

Host timing is useful for regression but cannot be generalized to the user's laptop before measurement.

Measurement	Work	Observed	Boundary
Native initial perft depth 5	4,865,609 nodes	1.344 s	3.620 Mnodes/s
Maximum RSS during captured depth-5 run	1,800 KiB	Linux container	host-specific
Packed differential	4 positions / 97 moves	0 mismatches	CPU fallback
Tablebase transport	296,674 bytes	2 exact files	hash verified
Python wheel	332,019 bytes	isolated install	PASS

24.1 What is not claimed

- No Stockfish, Leela Chess Zero or other mature-engine strength comparison.
- No RTX speedup, positions/s, energy efficiency, thermals, battery life or sustained laptop throughput.
- No implication that faster move generation changes the mathematical WDL obligations.
- No universal bit-identical floating-point behavior; promoted legality, hashes and proof records use exact integer/serialized boundaries.
- No proof that the Python interpreter, C++ compiler, operating system or GPU hardware is free of defects.

EQUAL-OUTPUT RULE

A performance claim becomes relevant only after CPU and CUDA outputs agree under the same rule profile and checker. Correctness is not inferred from speed.

25. Promotion Gates, Roadmap and Kill Criteria

The campaign is designed to fail safely and visibly when evidence is incomplete.

25.1 Promotion sequence

1. Freeze legal and history authority; maintain cross-implementation perft and transition tests.
2. Compile and differential-test SM120 kernels on the target laptop.
3. Add append-only frontier/proof storage and crash-safe checkpoints.
4. Expand exact endgame partitions with rule-profile and hash commitments.
5. Implement complete dead-position certificates or preserve UNKNOWN.
6. Close one root shard at a time; diversify independent checkers.
7. Promote the initial node only when its complete quantified WDL rule is satisfied.

25.2 Release-blocking failures

- Any legal-move or terminal mismatch, including castling, en-passant, promotion, check or draw-order errors.
- A proof verifier accepts a missing defender reply, illegal edge, wrong pre-state, mismatched hash or unsupported draw closure.
- A 64-bit key, score, GPU hash or cache hit is treated as complete proof identity.
- Repetition history, draw-claim ownership, dead-position scope or 50/75-move policy is implicit.
- A worker directly changes authoritative WDL without an independent checker record.
- Compression or seed reconstruction discards information needed to reproduce legal actions or certificates.
- The UGTS wrapper becomes less auditable than the conventional exact rule/proof implementation it is meant to discipline.

STOP CONDITION

When a boundary cannot be certified, the correct output is UNKNOWN plus a reason-coded open obligation. That is a successful safety behavior, not a reason to fabricate a value.

26. Chess Mechanism Catalog C001-C104

Namespaced mechanism records provide traceability without renumbering the broader UGTS catalogs.

Domain	Count	ID span	Representative records
Proof	13	C042-C079	OR attacker node; AND defender node; Checkmate leaf; ...
Tablebase	8	C048-C058	KXK exact profile; Validity cell; Terminal initialization; ...
Campaign	7	C080-C086	Twenty root obligations; Root WDL aggregate; SQLite obligation ledger; ...
GPU	7	C089-C097	CUDA fixed-point kernel; GPU proposal-only authority; Batch move protocol; ...
State	7	C003-C072	Position tuple; Side-to-move bit; Castling-right mask; ...
Search	6	C036-C041	Alpha-beta negamax; Iterative deepening; Quiescence; ...
Validation	5	C033-C103	Perft oracle; Automated release gates; Python/C++ perft differential; ...
Rules	4	C066-C069	Claim-as-action semantics; Intended-move draw claim; Fivefold automatic guard; ...
Terminal	4	C028-C031	Checkmate relation; Stalemate relation; Dead-position subset; ...
Guard	3	C018-C020	King-safety guard; Castling transit guard; En-passant exposure guard
Identity	3	C005-C070	Piece identity; Repetition identity; Legal en-passant repetition right
Packing	3	C049-C092	Dense 19-bit KXK key; 64-byte packed position; 16-bit move code
Transition	3	C023-C025	Deterministic board patch; Rights patch; Clock and turn patch
Encoding	2	C010-C034	Strict FEN codec; SAN projection
Event	2	C022-C026	Move proposal; Move event record
Governance	2	C002-C065	Evidence boundary; Component-scoped release identity
Native	2	C087-C088	C++20 exact rule oracle; Finite WDL fixed-point demo
Persistence	2	C011-C012	Canonical state record; SHA-256 state identity
Roadmap	2	C064-C104	Distributed global proof sharding; Initial-position proof promotion gate
Storage	2	C098-C099	Disk-backed proof DAG target; 64-bit index boundary
Branch	1	C021-C021	Promotion branch
Compatibility	1	C016-C016	Occupancy and color gate
Compression	1	C056-C056	Gzip table transport
Coordinates	1	C004-C004	Square address
Evidence	1	C059-C059	Tablebase metadata
Heuristic	1	C035-C035	Transparent evaluation
Indexing	1	C013-C013	Compact 64-bit cache key
Integration	1	C100-C100	External tablebase adapter gate
Presentation	1	C062-C062	Offline proof viewer
Profile	1	C001-C001	UGTS chess solver profile
Query	1	C032-C032	Legal move enumeration
Relation	1	C017-C017	Attack relation
Replay	1	C027-C027	Hash-checked replay
Schema	1	C061-C061	JSON contracts
Support	1	C015-C015	Piece movement support

Domain	Count	ID span	Representative records
Symmetry	1	C057-C057	Black-strong normalization
Tooling	1	C060-C060	Command-line workflow

26.1 Representative 2.0 additions

ID	Domain	Mechanism	Normalized definition
C065	Governance	Component-scoped release identity	Name the upgrade <code>ugts.application.chess-proof@2.0.0</code> rather than treating every UGTS number as one global ladder.
C069	Rules	Seventy-five-move automatic guard	Terminate at 150 reversible plies, with checkmate taking precedence.
C073	Proof	WDL four-value lattice	Use WIN, LOSS, DRAW and UNKNOWN; a cutoff is never coerced to DRAW.
C078	Proof	Content-addressed WDL node	Hash canonical state, profile, children, values and exactness into each bounded WDL node record.
C082	Campaign	SQLite obligation ledger	Store root jobs, status, leases, candidate certificate paths and verification records in a transactional database.
C087	Native	C++20 exact rule oracle	Provide a dependency-light native legality/perft/search/mate implementation independent of Python.
C092	Packing	16-bit move code	Pack source, destination and promotion into a reversible 16-bit value.
C097	GPU	Three-stream pipeline	Overlap upload, candidate expansion and download/checking through bounded streams without reordering proof commits.
C101	Validation	Python/C++ perft differential	Require exact move-path agreement across independent rule kernels on standard stress fixtures.
C104	Roadmap	Initial-position proof promotion gate	Promote the root from UNKNOWN only after all quantified WDL obligations, history semantics and checker records close.

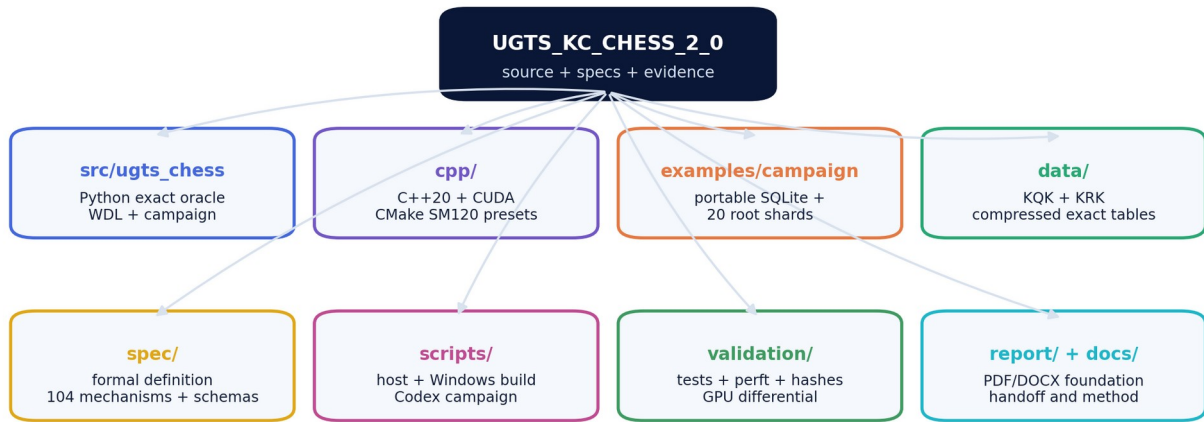
CATALOG MEANING

An ID records scope, implementation and validation linkage. It is not by itself a claim of scientific novelty, authorship priority, patentability or proof completeness.

27. Accompanying ZIP and Reproduction

The archive is a complete Codex handoff: source, exact data, schemas, campaign state, build scripts, report and captured evidence.

Accompanying ZIP: Complete Codex Handoff



Uploaded UGTS source PDFs are referenced by hash and not redistributed. Build artifacts are separated from source evidence.

Figure 19. Source PDFs are referenced by title, page basis and SHA-256 but are not redistributed.

Path	Contents
src/ugts_chess	Python rule/history oracle, WDL certificates, campaign, tablebases and CLI
cpp	C++20 core, host tests, packed protocol, CPU fallback and optional CUDA kernels
examples/campaign	SQLite root campaign, 20 shard JSON records and exact depth-4 workload checkpoint
data	KQK/KRK compressed exact tables and metadata
spec	Formal definition, schemas, source register, RTX profile and C001-C104 catalog
scripts	Codex task, Windows SM120 build and campaign run scripts
validation	Host tests, CTest, differential outputs, wheel, manifests and device placeholder
report / web	Editable DOCX, final PDF, diagrams and offline mate-proof viewer

27.1 Host reproduction

```

cd UGTS_KC_CHESS_2_0_RTX5070TI
PYTHONPATH=src python -m unittest discover -s tests -v
bash scripts/build_host.sh
PYTHONPATH=src python tools/run_validation_v2.py
python tools/generate_report_assets.py
python tools/build_report.py
  
```

28. Final Upgraded Definition

The foundation is a proof campaign with optional accelerators, not a search score presented as a solved game.

FORMAL DEFINITION

UGTS Chess Proof Campaign 2.0 is a finite, content-addressed application profile in which complete classical positions and exact repetition context define legal actions; special rights and king safety certify events; immutable transitions update state; claimable draws remain player actions; automatic terminals seed four-valued WDL obligations; certificates record existential witnesses or complete universal coverage; an independent checker promotes results into a hash-chained campaign ledger; exact KQK/KRK tables close bounded partitions; and C++20/CUDA adapters accelerate proposals and fixed-point work without becoming final proof authority.

28.1 Canonical object

```
UGTS-CHESS-2 = (Q, A, M, G, T, H, V, C, P, L, X)
Q complete typed state      A attack/support relations
M deterministic legal moves G terminal and draw guards
T immutable transitions     H exact history context
V WIN|LOSS|DRAW|UNKNOWN    C certificates/checkers
P campaign partitions       L replay/proof lineage
X optional CPU/CUDA adapters
```

28.2 Current root theorem

```
initial_position_value = UNKNOWN
reason = 20 exact root obligations exist; 0 child WDL certificates have been independently promoted
```

RELEASE INTERPRETATION

The package materially advances an attempt by making the open proof explicit, partitioned, portable and executable. It does not overstate that progress as a game-theoretic solution.

Appendix. Source Register, External References and Attribution

Supplied artifacts and current standards are separated from engineering additions and captured package evidence.

ID	Supplied source	Page basis	SHA-256	Use
S1	UGTS-GN 1.1	pp. 2-5, 9-15	3a6b6c57118ca4e2...	Canonical support, compatibility, guard, event, transition, lineage; precision and compression boundaries
S2	UGTS-KC 2.0	pp. 3-5, 8-15	a29dd9959ebd300c...	Typed state, query-first patterns, topology, hybrid events, bounded dynamics
S3	KC Two Hands 3.0	pp. 5-6, 11-16, 19-20	183ffa4e2bf29cfa...	Proposal verification, deterministic commit, state hashes, checkpoints and replay
S4	UGTS-KC 3.6	pp. 3-7, 9-14	e21433cd8378368d...	Definitions as data, content addresses, dependency closure and explicit operation order
S5	UGTS-KC 3.6.2 SCLP	pp. 7-15	6987490a0a7b7ed7...	Finite packed keys, radix tries, bounded branching and storage audits
S6	UGTS-KC 3.9	pp. 3-4, 8-13	d7ee3c91abf60e35...	Deterministic game world, canonical snapshots/hashes, CLI and offline HTML5 delivery
S7	UGTS-KC 3.6.3 SARA	pp. 1-2, 6-12	c5a8256e345047d5...	Reviewed but excluded as wallet/security-specific; only its explicit authorization boundary is retained as a governance analogy
S8	UGTS-KC 4.2 Go Solver	pp. 1-13 and appendices	1ea7d15b0112a870...	Board-game precedent for rules/position/game-solved separation, exact finite WDL certificates and explicit UNKNOWN boundaries
S9	UGTS-KC Chess 1.0	pp. 2-21	f8bce74ffeedb4...	Immediate exact rule, bounded mate, KQK/KRK and proof-viewer baseline
S10	UGTS-KC 4.2 General Operator and Order Addendum	pp. 2-17 and appendices	413ef9568330e253...	Explicit parse/dependency/evaluation/certification/event/replay order and fixed-point operator contracts
S11	UGTS Versioning Governance Phase 1	pp. 1-8	1b64495ed32a24a6...	Component-scoped release identity and explicit relationship graph

External standards and current device references

ID	Reference	Edition	Use	Location
E1	FIDE Laws of Chess	taking effect from 1 January 2023	Classical move, terminal, repetition and move-count semantics	https://handbook.fide.com/chapter/E012023
E2	NVIDIA GeForce RTX 50 Series Laptop specifications	current online source	RTX 5070 Ti Laptop core count, memory and Blackwell device family	https://www.nvidia.com/en-us/geforce/laptops/50-series/
E3	NVIDIA CUDA GPU Compute Capability	current online source	Runtime/compile architecture reference; GeForce RTX 5070 Ti listed under compute capability 12.0	https://developer.nvidia.com/cuda/gpus
E4	CUDA Toolkit Release Notes	current 13.x compatibility table consulted 29 August 2026	Toolkit/driver compatibility and Windows driver-separation boundary	https://docs.nvidia.com/cuda/cuda-toolkit-release-notes/
E5	Syzygy tablebases	current online source	External exact endgame adapter context; files not bundled	https://syzygy-tables.info/

Attribution and legal boundary

Prepared for Tom Klootwijk. The requester attribution is recorded as supplied and has not been independently verified. This report and package are technical design and validation artifacts. They are not legal proof of identity, authorship, ownership, priority, patentability, exclusive rights, licensing status or scientific consensus.

CLOSING STATEMENT

The strongest evidence in this release is exact and local: legal transitions, bounded certificates, finite tablebases, campaign hashes and passing reproducibility gates. The global classical root remains UNKNOWN until the missing proof obligations are actually closed.